

► Build a calculator that stays on the line

The task

any language, no framework, just a socket

```
GET /add?a=2&b=3    -> 200    5
GET /sub?a=10&b=4   -> 200    6
GET /mul?a=6&b=7    -> 200   42
GET /div?a=9&b=3    -> 200    3
```

```
GET /div?a=1&b=0    -> 400
GET /add?a=x&b=3    -> 400
GET /pow?a=2&b=8    -> 404
POST /add           -> 405
GET /add (no Host)  -> 400
```

That is the entire feature set. You could write it in an afternoon, and the arithmetic is not the point.

Don't read 02-http11/server11.py

How I will mark it

one socket. every request.

```
s = socket.create_connection(("localhost", 8080))
```

```
GET /add?a=2&b=3    -> 200    5
GET /sub?a=10&b=4   -> 200    6
GET /mul?a=6&b=7    -> 200   42
GET /div?a=1&b=0    -> 400
GET /pow?a=2&b=8    -> 404
POST /add           -> 405
```

```
socket still open: True
1 TCP handshake, 6 responses
```

~~If the socket dies before I am done,~~

The part that is actually hard. Keeping the connection open forces a question HTTP/1.0 never asked you: **where does this request end and the next one begin?** While you were hanging up, the answer was "at EOF", for free. Now you must consume exactly Content-Length bytes and not one more — byte n+1 belongs to somebody else.

Stretch, all optional: honour Connection: close · an idle timeout you can defend · chunked encoding · take all six at once and answer in order, which is pipelining.

► Now you write the spec

Track 1 — the server

```
$ ./bserve ./www 9000
```

```
accept a TCP connection
read one binary request frame
map the path to a file under a root
reply: status, headers, the bytes
404 if it is not there
400 if the frame is malformed
and keep the connection open
```

Track 2 — the client

```
$ ./bcurl -v localhost:9000/index.html
```

```
build the binary request frame
read the response, body to stdout
-v hexdumps every frame
exit non-zero on 4xx / 5xx
```

and never open a second connection

The bit in the middle is the actual project

A fixed-size frame header — you pick the fields and the widths, and you defend them. HTTP/2 chose 24 / 8 / 8 / 31. Why?

Headers: number the ten names you actually send, length-prefix the rest — HPACK's first two mechanisms, in an evening.

And one line you may not skip: a receiver meeting a frame type it does not know MUST skip it cleanly. That is how you leave room for a version 2.

What you hand in

1. The spec. Two pages. Enough for a stranger.
2. Your program.
3. An annotated hexdump of one complete request and response.

If you cannot annotate your own bytes, the spec is not finished.

In pairs: one server, one client, and the only thing that crosses between you is the spec. A client that only works against your own server is an implementation, not a protocol.